

Project - Low Level Design on CI/CD for Go (Golang) Logistics App

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	JIYANSHI KESHRI	EN22CS303026
2	MANASVI JAIN	EN22CS301580
3	KRATIKA TOMAR	EN22CS301516
4	MADHURI PRAJAPATI	EN22CS301562
5	NIKHIL KUMAR SINGH	EN22CS301648
6	PRIYANKA KUMARI	EN22CS301758

Group Name: Placed Btech Group 05

Project Number: DO-05

Industry Mentor Name:

University Mentor Name:

Academic Year: 2026

1. Introduction

This document explains the **Low-Level Design (LLD)** of the *CI/CD-enabled Go Logistics Application*.

The main goal of this project is to:

- Build a simple logistics web application using Go
- Use **Docker** for containerization
- Ensure the application runs the same way on all systems (no environment issues)

It also describes:

- How the application works internally
- How Docker is used to build and run the app
- How different components interact with each other

1.1 Scope of the document

This document includes:

- Docker-based containerization of the Go application
- Internal working and execution flow of the system
- Build process and runtime configuration
- Design of the application inside a container

It does not include production deployment or advanced orchestration.

1.2 Intended Audience

This document is useful for:

- DevOps Engineers – to understand containerization and CI/CD flow
- Software Developers – to understand application design and deployment
- System Administrators – to manage and run containers
- Project Mentors – to review system design

- Students – who are learning CI/CD and Docker.

1.3 System overview

The Logistics Application is a lightweight web service written in Go. It is containerized using Docker to ensure environment consistency. The system provides HTTP response endpoints and simulates logistics service availability.

2 System Design

The system is divided into three main layers:

1. **Go Application Layer** – where the actual code runs
2. **Docker Container Layer** – where the app is packaged and executed
3. **Host Environment** – the system (laptop/server) running Docker

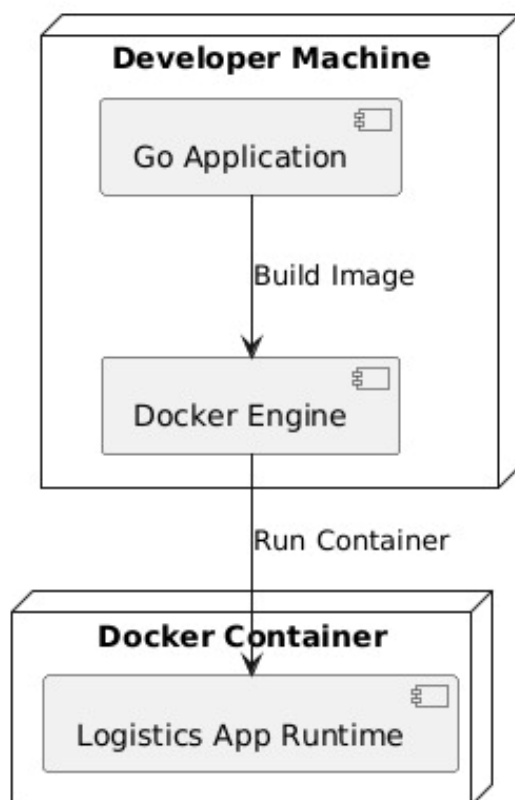


Fig.1 UML – System Architecture

2.1 Application Design

Application is a simple HTTP server handling logistics requests.

Main Components:

- HTTP Handler
- Server Listener
- Runtime Binary

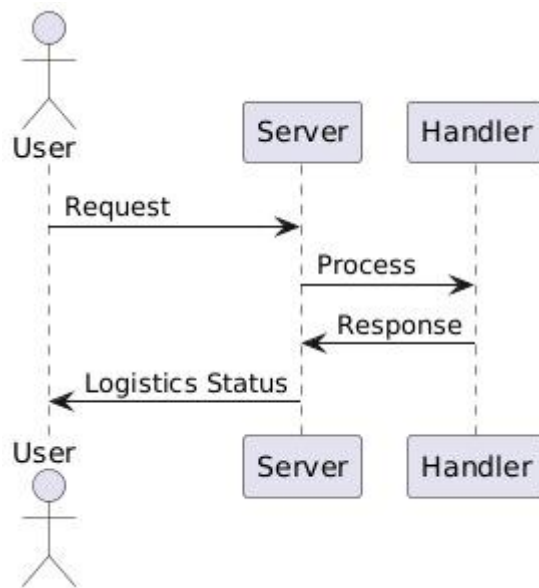


Fig.2 UML - Application Flow

2.2 Process Flow

Step-by-step workflow:

1. Developer writes the Go application code.
2. A Dockerfile is created to define how the app should be built.
3. Docker builds an image using the Dockerfile.
4. The image is used to create and run a container.
5. The container starts the application.
6. The user accesses the app through a browser.

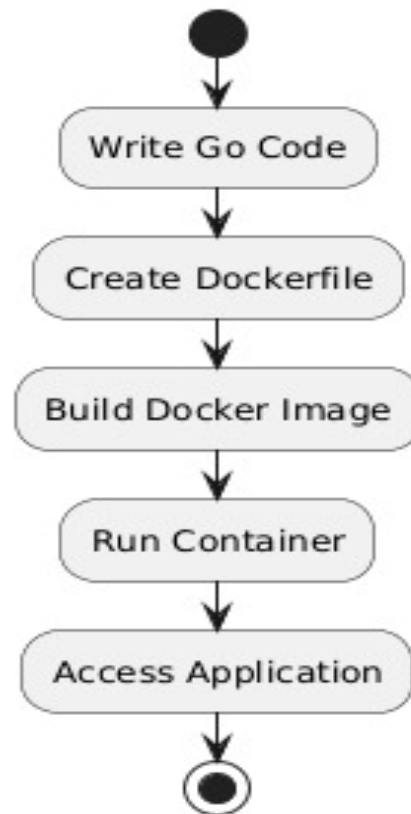


Fig.3 UML – Process Flow

2.3 Information Flow.

Information flows from:

User → HTTP Request → Container → Application → Response → User

2.4 Components Design

1. Go Application
2. Docker Image
3. Docker Container
4. Host Machine

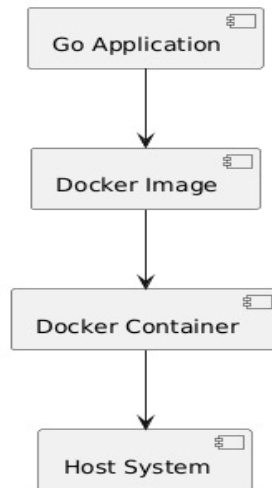


Fig.4 UML Component Diagram

2.5 Key Design Considerations

- Environment consistency
- Fast deployment
- Lightweight containerization
- Scalability readiness
- Platform independence

2.6 API Catalogue

- `GET /` → Returns welcome message
- `GET /status` → Shows service status
- `GET /health` → Checks if application is running

3 References

Docker Official Docs

Go Official Documentation

DevOps CI/CD Best Practices